Transformer_and_attention_sink

2026 年 4 月 20 日

1 Transformer block 的工作流程

设输入为张量 $\mathbf{X}$（维度为 `[batch_size, sequence_length, d_model]`）

1.1 第一次归一化 Pre-RMSNorm

先稳住方差，防止计算爆炸。

$$\mathbf{X}_{norm1} = \mathrm{RMSNorm}(\mathbf{X})$$

其中

$$\mathrm{RMSNorm}(\mathbf{h}) = \frac{\mathbf{h}}{\sqrt{\frac{1}{d_{\text{model}}}\sum_{i=1}^{d_{\text{model}}} h_i^2}}$$

1.2 全局信息交互 Multi-Head Attention (MHA)

词与词之间交换视野，提取上下文关联。

$$\mathbf{H} = \mathrm{MultiHeadAttention}(\mathbf{X}_{norm1})$$

1.2.1 线性投影与分头

将 $\mathbf{X}_{norm1}$ 分别乘以三个可学习的权重矩阵 $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$，得到 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$。然后将 `d_model` 维度切分成 `h` 个头。例如，`d_model` 为 512，`h` 为 8，那么每个头的维度 `d_k` 就是 64。张量变形为 `[batch, h, seq_len, d_k]`。

1.2.2 旋转位置编码 RoPE

空间的切片 假设我们有一个当前位置为 m 的 Query 向量 $\mathbf{q}$，它的特征维度是 `d_k`（以下简记为 d）。

把这 d 维空间两两配对，切分成 d/2 个相互独立的二维子空间：* 子空间 1：[q_0, q_1] * 子空间 2：[q_2, q_3] * … * 子空间 d/2：[q_{d-2}, q_{d-1}]

乘旋转矩阵 在每个 2 维子空间上，我们应用一个旋转矩阵，旋转角度与位置 m 和维度 i 相关：

$$\begin{bmatrix} q'_{2i} \\ q'_{2i+1} \end{bmatrix} = \begin{bmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{bmatrix} \begin{bmatrix} q_{2i} \\ q_{2i+1} \end{bmatrix}$$

其中 $\theta_i = b^{-\frac{2i}{d}}, \quad i \in \{0, 1, \dots, \frac{d}{2} - 1\}$ ，b 是一个常数（通常取 10000）。

理论计算流程

1. 原始向量 $\mathbf{q} = [q_0, q_1, q_2, q_3, ..., q_{d-2}, q_{d-1}]$
2. 正负翻转向量 $\mathbf{q}_{flip} = [-q_1, q_0, -q_3, q_2, ..., -q_{d-1}, q_{d-2}]$
3. 旋转算子

- $\mathbf{cos} = [\cos(m\theta_0), \cos(m\theta_0), \cos(m\theta_1), \cos(m\theta_1), \cdots, \cos(m\theta_{d/2-1}), \cos(m\theta_{d/2-1})]$
- $\mathbf{sin} = [\sin(m\theta_0), \sin(m\theta_0), \sin(m\theta_1), \sin(m\theta_1), \cdots, \sin(m\theta_{d/2-1}), \sin(m\theta_{d/2-1})]$

4. 最终旋转结果 $\mathbf{q}_{out} = \mathbf{q} \odot \mathbf{cos} + \mathbf{q}_{flip} \odot \mathbf{sin}$

工业计算流程（为了计算速度） q_0 **和** $q_{d/2}$ **是一对，**q_1 **和** $q_{d/2+1}$ **是一对，以此类推。** 1. 原始向量 $\mathbf{q} = [q_0, q_1, q_2, q_3, \cdots, q_{d-2}, q_{d-1}]$ 2. 正负翻转向量 $\mathbf{q}_{flip} = [-q_{d/2}, -q_{d/2+1}, \cdots, -q_{d-1}, q_0, q_1, \cdots, q_{d-1}]$ 3. 旋转算子 - $\mathbf{cos} = [\cos(m\theta_0), \cos(m\theta_1), \cdots, \cos(m\theta_{d/2-1}), \cos(m\theta_0), \cos(m\theta_1), \cdots, \cos(m\theta_{d/2-1})]$ - $\mathbf{sin} = [\sin(m\theta_0), \sin(m\theta_1), \cdots, \sin(m\theta_{d/2-1}), \sin(m\theta_0), \sin(m\theta_1), \cdots, \sin(m\theta_{d/2-1})]$ 4. 最终旋转结果 $\mathbf{q}_{out} = \mathbf{q} \odot \mathbf{cos} + \mathbf{q}_{flip} \odot \mathbf{sin}$

同样的操作也应用于 Key 向量 $\mathbf{K}$。

代码

[3]:
```
import torch
import torch.nn as nn
class RoPE(nn.Module):
    def __init__(self, d_k, max_seq_len = 4096, b=10000):
        super().__init__()
        theta_i = 1/(b**(torch.arange(0,d_k,2).float()/d_k))  # \theta_i =␣
↪b^{-\frac{2i}{d}}, \quad i \in \{0,1,\dots,\frac{d}{2}-1\}
        m = torch.arange(max_seq_len).float()  # m = [0,1,...,seq_len-1]
        m_theta_i = torch.outer(m, theta_i)  # [seq_len, d_k/2]
        cos = torch.cos(torch.cat((m_theta_i, m_theta_i), dim=-1))  # [seq_len,␣
↪d_k]
```

```
        sin = torch.sin(torch.cat((m_theta_i, m_theta_i), dim=-1))  # [seq_len,␣
↪d_k]
        self.register_buffer('cos', cos[None, None, :, :])  # 好处: cos 和 sin
不需要更新参数，注册为 buffer 后会自动放到正确的设备上
        self.register_buffer('sin', sin[None, None, :, :])  # 扩充维度以适应后续
计算

    def forward(self, x): # 适用于 Q、K (V 不需要位置编码!)
        # x: [batch_size, num_heads, seq_len, d_k]
        seq_len = x.shape[2]
        d_2 = x.shape[-1] // 2
        cos = self.cos[:, :, :seq_len, :]  # [1, 1, seq_len, d_k]
        sin = self.sin[:, :, :seq_len, :]
        x_first_half = x[..., :d_2]
        x_second_half = x[..., d_2:]
        x_flip = torch.cat((-x_second_half, x_first_half), dim=-1)
        x_out = x * cos + x_flip * sin  # 旋转位置编码
        return x_out
```

1.2.3 缩放点积注意力与因果掩码 Causal Masking

Query-Key 点积计算注意力得分：

$$\textbf{Scores} = \frac{\mathbf{QK}^T}{\sqrt{d_k}}$$

得到一个 `[batch_size, h, seq_len, seq_len]` 的得分矩阵。

因果掩码和 softmax 计算注意力权重 把上三角 $-\infty$ 掩码矩阵

$$\textbf{Mask} = \begin{bmatrix} 0 & -\infty & -\infty & \cdots \\ 0 & 0 & -\infty & \cdots \\ 0 & 0 & 0 & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

加到 **Scores** 上，遮蔽未来词并使用 softmax 计算注意力权重：

$$\mathbf{A} = \mathrm{Softmax}(\textbf{Scores} + \textbf{Mask})$$

其中

$$\mathrm{Softmax}(\mathbf{x})_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

注：不能乘 1 和 0 的掩码，因为后面要进行 softmax，$e^{-\infty}$ 才能变成 0。

乘以 Value 完成加权求和 最后，将注意力权重与 $\mathbf{V}$ 相乘，完成全局信息的加权坍缩：

$$\mathbf{Out} = \mathbf{AV}$$

1.2.4 拼接与输出

将 h 个头的结果在最后一个维度上拼起来（Concat），恢复到 d_model 维度，得到矩阵 $\mathbf{Out}'$，最后乘以一个输出矩阵 $\mathbf{W}_O$ 进行线性映射

$$\mathbf{H} = \mathbf{Out}'\mathbf{W}_O$$

1.2.5 代码

[4]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F
class MultiHeadAttention(nn.Module):
    def __init__(self, num_heads, d_model, max_seq_len=4096):
        super().__init__()
        assert d_model % num_heads == 0, "d_model must be divisible by␣
↪num_heads"
        # 维度属性
        self.num_heads = num_heads
        self.d_model = d_model
        self.d_k = d_model // num_heads
        # 权重矩阵
        self.W_Q = nn.Linear(d_model, d_model, bias=False)
        self.W_K = nn.Linear(d_model, d_model, bias=False)
        self.W_V = nn.Linear(d_model, d_model, bias=False)
        self.W_O = nn.Linear(d_model, d_model, bias=False)
        # RoPE 位置编码
        self.rope = RoPE(self.d_k, max_seq_len=max_seq_len)
        # 因果掩码
        tril = torch.tril(torch.ones(max_seq_len, max_seq_len)).bool() # torch.
↪tril (triangular lower) 提取下三角元素，并把上三角元素置 0
        mask = torch.zeros(max_seq_len, max_seq_len).masked_fill(~tril,␣
↪float('-inf')) # 下三角元素置 0，上三角元素置-inf
        self.register_buffer('mask', mask[None, None, :, :])  # [1, 1, seq_len,␣
↪seq_len]
```

```python
    def forward(self, x): # x: [batch_size, seq_len, d_model]
        # 线性投影与分头
        batch_size, seq_len, _ = x.shape
        Q = self.W_Q(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)  # [batch_size, num_heads, seq_len, d_k]
        K = self.W_K(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)  # 交换 (1, 2) 是因为之后 softmax 时要进行矩阵乘法 (只乘后两维)
        V = self.W_V(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)  # 在 seq_len 维度上分配注意力
        # RoPE 位置编码
        Q = self.rope(Q)
        K = self.rope(K)
        # 缩放点积注意力与因果掩码 Causal Masking
        ## Query-Key 点积计算注意力得分
        scores = torch.matmul(Q, K.transpose(-2, -1)) / (self.d_k ** 0.5)  # [batch_size, num_heads, seq_len, seq_len]
        ## 因果掩码和 softmax 计算注意力权重
        Attention = F.softmax(scores + self.mask[..., :seq_len, :seq_len], dim=-1)  # [batch_size, num_heads, seq_len, seq_len]
        ## 乘以 Value 完成加权求和
        Out = torch.matmul(Attention, V)  # [batch_size, num_heads, seq_len, d_k]
        # 多头拼接并乘以输出矩阵 (必须先内存连续化 (contiguous) 再 view, 否则会报错)
        Out = Out.transpose(1, 2).contiguous().view(batch_size, seq_len, -1)  # [batch_size, seq_len, d_model]
        H = self.W_O(Out)  # [batch_size, seq_len, d_model]
        return H
```

1.3 第一次残差连接

保留原始信息，叠加全局视野

$$\mathbf{X}_1 = \mathbf{X} + \mathbf{H}$$

注：不要写成 $\mathbf{X}_1 = \mathbf{X}_{norm1} + \mathbf{H}$！

1.4 第二次归一化

$$\mathbf{X}_{norm2} = \mathrm{RMSNorm}(\mathbf{X}_1)$$

1.5 局部特征升维与非线性映射 Feed-Forward Network (FFN)

$$\mathbf{X}_{out} = \mathrm{SwiGLU}(\mathbf{X}_{norm2})$$

其中

$$\mathrm{SwiGLU}(x) = [\mathrm{Swish}(xW) \odot (xV)]W_2$$

[5]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F
class SwiGLU(nn.Module):
    def __init__(self,d_model,d_hidden):
        super().__init__()
        self.W = nn.Linear(d_model,d_hidden,bias=False)
        self.V = nn.Linear(d_model,d_hidden,bias=False)
        self.W2 = nn.Linear(d_hidden,d_model,bias=False)
    def forward(self,x):
        x1 = F.silu(self.W(x))
        x2 = self.V(x)
        x_out = self.W2(x1 * x2)
        return x_out
```

1.6 第二次残差连接

这就是当前 Block 的最终输出，随后原封不动地送入下一个 Block

$$\mathbf{X}_2 = \mathbf{X}_1 + \mathbf{X}_{out}$$

注：不要写成 $\mathbf{X}_2 = \mathbf{X}_{norm2} + \mathbf{X}_{out}$！

1.7 代码拼装

[6]:
```
class TransformerBlock(nn.Module):
    def __init__(self, num_heads, d_model, max_seq_len=4096, multiplier=4):
        super().__init__()
        # 用 RMSNorm 类而不是函数，因为它自动包含了可学习的缩放参数
        self.rmsnorm1 = nn.RMSNorm(d_model)  # 第一次归一化 Pre-RMSNorm
        self.rmsnorm2 = nn.RMSNorm(d_model)  # 第二次归一化
        self.attention = MultiHeadAttention(num_heads, d_model,␣
 ↪max_seq_len=max_seq_len)
        self.ffn = SwiGLU(d_model, d_model * multiplier)

    def forward(self, x):
        # 第一次归一化 Pre-RMSNorm
        x_norm1 = self.rmsnorm1(x)
        # 全局信息交互 Multi-Head Attention (MHA)
        h = self.attention(x_norm1)
        # 第一次残差连接
        x1 = x + h
        # 第二次归一化
        x_norm2 = self.rmsnorm2(x1)
        # 前馈网络 Feed-Forward Network (FFN)
        x_out = self.ffn(x_norm2)
        # 第二次残差连接
        x2 = x1 + x_out
        return x2
```

2 用 Toy Model 探究 Attention Sink

内容：按理论搭建几层相同权重的 Transformer 的 toy model，并观测 attention 的变化，看看什么情况下出现 attention sink 与 massive attention

2.1 Toy Model 搭建

[205]:
```
class ToyModel(nn.Module):
    def __init__(self, num_blocks, num_heads=8, d_model=512, max_seq_len=4096):
        super().__init__()
```

```
        self.transformer_block = TransformerBlock(num_heads, d_model,␣
↪max_seq_len=max_seq_len) # 这里保证了各层权重始终相同
        self.num_blocks = num_blocks

    def forward(self, x):
        for _ in range(self.num_blocks):
            x = self.transformer_block(x)
        return x
```

2.1.1 PyTorch Hook

1. 探头函数（Hook Function）的规范：

- 对于前向传播的 Hook，PyTorch 规定你的回调函数必须严格接收三个参数：
  - module：当前被挂载的网络层对象（比如某个 Linear 层）。
  - input：一个元组（Tuple），包含了流入该层的所有张量。
  - output：该层计算完毕、正准备传给下一层的输出张量（Tensor）。这就是我们要截获的“脏水”或者“激增信号”。

2. 挂载动作与解除：

- 任何继承自 nn.Module 的对象都有一个自带的方法.register_forward_hook(hook_fn)。调用它，探头就夹上去了。这个方法还会返回一个句柄（Handle），如果你想拆掉探头，只需调用 handle.remove()。

[206]:
```
class AttentionProbe:
    def __init__(self):
        self.captured_data = []
    def __call__(self, module, input, output):
        attention = module.captured_attention.detach().cpu().numpy()
        self.captured_data.append(attention)
    def reset(self):
        self.captured_data = []
```

为了探测 Attention 矩阵（`[batch_size, num_heads, seq_len, seq_len]`），我们需要微调 Multi-Head Attention

[7]:
```
class MultiHeadAttention(nn.Module):
    def __init__(self, num_heads, d_model, max_seq_len=4096):
```

```
        super().__init__()
        assert d_model % num_heads == 0, "d_model must be divisible by␣
↪num_heads"
        # 维度属性
        self.num_heads = num_heads
        self.d_model = d_model
        self.d_k = d_model // num_heads
        # 权重矩阵
        self.W_Q = nn.Linear(d_model, d_model, bias=False)
        self.W_K = nn.Linear(d_model, d_model, bias=False)
        self.W_V = nn.Linear(d_model, d_model, bias=False)
        self.W_O = nn.Linear(d_model, d_model, bias=False)
        # RoPE 位置编码
        self.rope = RoPE(self.d_k, max_seq_len=max_seq_len)
        # 因果掩码
        tril = torch.tril(torch.ones(max_seq_len, max_seq_len)).bool() # torch.
↪tril (triangular lower) 提取下三角元素，并把上三角元素置 0
        mask = torch.zeros(max_seq_len, max_seq_len).masked_fill(~tril,␣
↪float('-inf')) # 下三角元素置 0，上三角元素置-inf
        self.register_buffer('mask', mask[None, None, :, :])  # [1, 1, seq_len,␣
↪seq_len]

    def forward(self, x): # x: [batch_size, seq_len, d_model]
        # 线性投影与分头
        batch_size, seq_len, _ = x.shape
        Q = self.W_Q(x).view(batch_size, seq_len, self.num_heads, self.d_k).
↪transpose(1, 2)  # [batch_size, num_heads, seq_len, d_k]
        K = self.W_K(x).view(batch_size, seq_len, self.num_heads, self.d_k).
↪transpose(1, 2)  # 交换 (1, 2) 是因为之后 softmax 时要进行矩阵乘法 (只乘后两维)
        V = self.W_V(x).view(batch_size, seq_len, self.num_heads, self.d_k).
↪transpose(1, 2)  # 在 seq_len 维度上分配注意力
        # RoPE 位置编码
        Q = self.rope(Q)
        K = self.rope(K)
        # 缩放点积注意力与因果掩码 Causal Masking
        ## Query-Key 点积计算注意力得分
```

```
        scores = torch.matmul(Q, K.transpose(-2, -1)) / (self.d_k ** 0.5)  #␣
↪[batch_size, num_heads, seq_len, seq_len]
        ## 因果掩码和 softmax 计算注意力权重
        Attention = F.softmax(scores + self.mask[..., :seq_len, :seq_len],␣
↪dim=-1)  # [batch_size, num_heads, seq_len, seq_len]
        self.captured_attention = Attention  # ** 新增 **: 捕获注意力矩阵，用
hook 记录下来，方便后续分析
        ## 乘以 Value 完成加权求和
        Out = torch.matmul(Attention, V)  # [batch_size, num_heads, seq_len,␣
↪d_k]
        # 多头拼接并乘以输出矩阵 (必须先内存连续化 (contiguous) 再 view，否则会报
错)
        Out = Out.transpose(1, 2).contiguous().view(batch_size, seq_len, -1)  #␣
↪[batch_size, seq_len, d_model]
        H = self.W_O(Out)  # [batch_size, seq_len, d_model]
        return H
```

把 hook 挂在 ToyModel 的 TransformerBlock 中的 Multi-Head Attention 上，这样就能捕获每一层的注意力矩阵，方便后续分析。

[208]:
```
class ToyModel(nn.Module):
    def __init__(self, num_blocks, num_heads=8, d_model=512, max_seq_len=4096):
        super().__init__()
        self.transformer_block = TransformerBlock(num_heads, d_model,␣
↪max_seq_len=max_seq_len) # 这里保证了各层权重始终相同
        self.num_blocks = num_blocks
        self.probe = AttentionProbe()
        self.transformer_block.attention.register_forward_hook(self.probe)
        self.captured_attention = None

    def forward(self, x):
        self.probe.reset() # 每次前向传播前重置 probe，清空上一次的观测数据
        for _ in range(self.num_blocks):
            x = self.transformer_block(x)
        self.captured_attention = list(self.probe.captured_data) # 显式 copy 一
份数据，避免后续被修改
```

```
        return x
```

2.1.2 Embedding (词嵌入，将离散的 token 转换为连续的向量表示)

vocab_size 表示词表大小（也就是模型的”词汇量”）

[209]:
```
class ToyModel(nn.Module):
    def __init__(self, num_blocks, num_heads=8, d_model=512, max_seq_len=4096,␣
 ↪vocab_size=10000):
        super().__init__()
        self.transformer_block = TransformerBlock(num_heads, d_model,␣
 ↪max_seq_len=max_seq_len) # 这里保证了各层权重始终相同
        self.num_blocks = num_blocks
        self.probe = AttentionProbe()
        self.transformer_block.attention.register_forward_hook(self.probe)
        self.captured_attention = None
        self.embedding = nn.Embedding(vocab_size, d_model)  # 新增：词嵌入层，将
离散的 token 转换为连续的向量表示
        # self.embedding 的作用：输入是离散的整数 (代表离散的 Token)，输出是连续的
embedding 向量 (每个 Token 对应一个 d_model 维的向量)，这个映射是可学习的

    def forward(self, input_ids): # input_ids 的形状为 [batch_size, seq_len]
        self.probe.reset() # 每次前向传播前重置 probe，清空上一次的观测数据
        x = self.embedding(input_ids) # 将输入的 token ids 转换为嵌入向量，形状
为 [batch_size, seq_len, d_model]
        for _ in range(self.num_blocks):
            x = self.transformer_block(x)
        self.captured_attention = list(self.probe.captured_data) # 显式 copy 一
份数据，避免后续被修改
        return x
```

2.1.3 分词器 (Tokenizer)

- LLM 的标准算法：BPE（Byte Pair Encoding）
  1. 初始化：把语料库（比如整个维基百科）拆成最基础的字符（或字节），这作为初始的微小词表。
  2. 统计合并：代码自动扫描整个语料，寻找相邻出现频率最高的配对。比如发现 t 和 h 经常连在一起，就把它们合并成一个新 Token th 加入词表。

3. 迭代生长：接着发现 th 和 e 经常连在一起，就合并出 the。发现 a 和 p 和 p 和 l 和 e 经常在一起，就合并出 apple。
4. 停止机制：当词表大小达到预设值（比如指定的 vocab_size = 10000）时，算法停止。

简易分词器

[210]:
```
class SimpleTokenizer:
    def __init__(self, corpus): # 输入语料文本
        self.chars = ['<pad>'] + sorted(list(set(corpus))) # 把用于填充空位的特殊 token <pad> 放在第 0 位，确保它的 id 为 0
        self.vocab_size = len(self.chars)
        self.char_to_id = {char:id for id, char in enumerate(self.chars)}
        self.id_to_char = {id:char for id, char in enumerate(self.chars)}

    def encode(self, texts):
        if isinstance(texts, str):
            texts = [texts]
        ids = [[0]+[self.char_to_id[char] for char in text] for text in texts]␣
 ↪# 在每个文本的开头加一个 <pad> token
        # 填充 0 使得所有序列长度相同，这样才能转化为 tensor
        max_len = max(len(id_seq) for id_seq in ids)
        batch_size = len(ids)
        padded_ids = torch.zeros((batch_size, max_len),dtype=torch.long)  # 使用 0 进行填充，因为 0 对应 <pad> token
        for i, id_seq in enumerate(ids):
            padded_ids[i, :len(id_seq)] = torch.tensor(id_seq)
        return padded_ids

    def decode(self, id_seqs):
        if isinstance(id_seqs, torch.Tensor):
            id_seqs = id_seqs.tolist()
        texts = []
        for id_seq in id_seqs:
            chars = [self.id_to_char[id] for id in id_seq if id != 0] # 解码时忽略 <pad> token
            text = ''.join(chars)
            texts.append(text)
        return texts
```

2.1.4 预测头

一个线性映射，把 Transformer 输出的 [batch_size, seq_len, d_model] 的 x 转换成 [batch_size, seq_len, vocab_size] 的 logits，表示每个位置上预测下一个 token 的未归一化的对数概率分布。

[211]:
```
class ToyModel(nn.Module):
    def __init__(self, num_blocks, num_heads=8, d_model=512, max_seq_len=4096,␣
 ↪vocab_size=10000):
        super().__init__()
        self.transformer_block = TransformerBlock(num_heads, d_model,␣
 ↪max_seq_len=max_seq_len) # 这里保证了各层权重始终相同
        self.num_blocks = num_blocks
        self.num_heads = num_heads
        self.d_model = d_model
        self.max_seq_len = max_seq_len
        self.vocab_size = vocab_size
        self.probe = AttentionProbe()
        self.transformer_block.attention.register_forward_hook(self.probe)
        self.captured_attention = None
        self.embedding = nn.Embedding(vocab_size, d_model)  # 新增：词嵌入层，将
离散的 token 转换为连续的向量表示
        # self.embedding 的作用：输入是离散的整数 (代表离散的 Token)，输出是连续的
embedding 向量 (每个 Token 对应一个 d_model 维的向量)，这个映射是可学习的
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
        # 新增：预测头，将 [batch_size, seq_len, d_model] 的 x 转换成␣
 ↪[batch_size, seq_len, vocab_size] 的 logits，表示每个位置上预测下一个 token 的
未归一化的对数概率分布。

    def forward(self, input_ids): # input_ids 的形状为 [batch_size, seq_len]
        self.probe.reset() # 每次前向传播前重置 probe，清空上一次的观测数据
        x = self.embedding(input_ids) # 将输入的 token ids 转换为嵌入向量，形状
为 [batch_size, seq_len, d_model]
        for _ in range(self.num_blocks):
            x = self.transformer_block(x)
        self.captured_attention = list(self.probe.captured_data) # 显式 copy 一
份数据，避免后续被修改
```

```
        logits = self.lm_head(x) # [batch_size, seq_len, vocab_size]
        return logits
```

2.2 损失计算：Cross-Entropy Loss (交叉熵)

2.2.1 信息熵

Boltzmann 熵：$S = k \ln \Omega$，其中 Ω 是系统的微观状态数

单个事件 x 的信息量：$I(x_i) = -\log_2 P(x_i)$，其中 $P(x_i)$ 是事件 x_i 发生的概率

Shannon 熵：整个系统所有事件信息量的期望值，

$$H(P) = -\sum_i P(x_i) \log_2 P(x_i)$$

2.2.2 相对熵（KL 散度，Kullback-Leibler Divergence）

记 $P(x)$ 是真实分布，$Q(x)$ 是模型预测的分布。

如果我们知道真实的概率分布，那么信息量就是 Shannon 熵 $H(P)$。但是现实中，我们只有猜测的分布 Q，那么为事件 x_i 计算的信息量就变成了 $-\log_2 Q(x_i)$。

此时，系统的信息量变成了

$$-\sum_i P(x_i) \log_2 Q(x_i)$$

这就是交叉熵 $H(P, Q)$。

KL 散度就是这两种情况的差值：

$$D_{KL}(P \| Q) = \sum_i P(x_i) \log_2 \frac{P(x_i)}{Q(x_i)}$$

由此，交叉熵可以分解为 Shannon 熵和 KL 散度的和：

$$H(P, Q) = H(P) + D_{KL}(P \| Q)$$

2.2.3 计算 Loss

理论上，我们在训练过程中希望最小化 KL 散度 $D_{KL}(P \| Q)$

但由于 P 是我的真实训练数据，是固定的，所以 Shannon 熵 $H(P)$ 是一个常数。因此，最小化 KL 散度等价于最小化交叉熵 $H(P, Q)$。为了节省计算资源，我们直接最小化交叉熵就好了。

$$Loss = H(P, Q) = -\sum_i P(x_i) \log_2 Q(x_i)$$

另一方面，受限于观测手段，我们在训练集中拿到的只是真实分布 P 的一个样本，因此计算的 P 只能是一个 one-hot 向量（在正确的 token 位置上是 1，其他位置是 0）。在这种情况下，$H(P) = 0$，所以交叉熵 $H(P, Q)$ 就直接等于 $D_{KL}(P\|Q)$。

2.2.4 另一种视角：最大似然估计（Maximum Likelihood Estimation, MLE）

从概率视角看，我们要做的是最大化模型在数据集上 100% 准确的概率。

似然函数 L（即模型在训练数据上完全正确的概率）可以表示为：

$$L(\theta) = \prod_{i=1}^{N} Q(x_i|\theta)$$

其中 $Q(x_i|\theta)$ 是模型在参数 θ 下对样本 x_i 的预测概率。

但是，直接计算这个乘积会导致数值下溢（因为概率都是小于 1 的数，连续相乘会变得非常小）。因此，我们通常对似然函数取对数，得到对数似然函数：

$$\log L(\theta) = \sum_{i=1}^{N} \log Q(x_i|\theta)$$

而对于每个 one-hot 样本，如果答案是 x_j，那么 $P(x_i) = \delta_{ij}$，单次训练的 Loss 就变成了：

$$Loss_j = -\log Q(x_j)$$

于是，所有样本的总 Loss 就是：

$$Loss = -\sum_{j=1}^{N} \log Q(x_j)$$

因此，最大化似然函数就等价于最小化交叉熵 Loss！

2.2.5 在神经网络中的计算方法

在神经网络中，我们算出输出的 embedding 张量 $\mathbf{X}_{out}$，然后一个可学习线性映射映射到词表空间，得到 logits，它的取值范围是 $(-\infty, +\infty)$。

但是，我们想要一个和为 1 且各元素非负的概率分布，所以我们对 logits(记为 z) 应用 softmax 函数：

$$Q(x_i) = \text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

物理意义：logits 其实对应了“负能量” 在 Boltzmann 分布中，

$$Q(x_i) = \frac{e^{-\frac{E_i}{k_B T}}}{Z}$$

其中

$$Z = \sum_j e^{-\frac{E_j}{k_B T}}$$

于是 z_i 就对应了 $-\frac{E_i}{k_B T}$，也就是说，logits 实际上是“负能量”的表示。模型通过调整参数来降低正确答案的“能量”，从而提高它的概率。

2.2.6 完整的计算流程

1. 传入 model 的输出 `x_out`，经过一个 `nn.Linear` 映射到词表空间，变为 `logits`。
2. 使用 `torch.softmax` 将 `logits` 转换成概率分布 Q。
3. 使用 `torch.sum` 将 Q 和真实分布 P（one-hot 向量）相乘并求和取负，得到交叉熵 Loss
4. 反向传播公式：

$$\frac{\partial Loss}{\partial z_i} = Q(x_i) - P(x_i)$$

[212]:
```
class CrossEntropyLoss:
    def __init__(self):
        self.P = None  # 真实分布
        self.Q = None  # 预测分布

    def forward(self, logits, labels, smoothing=0.1):
        # logits: 形状为 (N,C),N 是批次大小，C 是类别数（词表大小）
        # labels: 形状为 (N,)，包含每个样本的真实类别索引
        P_manual = torch.zeros_like(logits)
        P_manual[torch.arange(logits.shape[0]), labels] = 1  # 将正确类别的位置设置为 1，其他位置为 0，形成 one-hot 分布
        self.P = P_manual * (1 - smoothing) + smoothing / logits.shape[1]  # 标签平滑处理
        self.Q = torch.softmax(logits, dim=-1)
        loss = -torch.sum(self.P * torch.log(self.Q + 1e-9)) / logits.shape[0] ␣
↪# 加上一个小常数避免 log(0)
        return loss

    def backward(self):
        grad = (self.Q - self.P) / self.P.shape[0]  # 取平均是因为 loss 是所有样本的平均损失，这样 loss 是一个强度量，grad 的数值范围也更合理，lr 就不需要根据批次大小进行调整了
        return grad
```

2.2.7 实际上做法

使用 PyTorch 的 `nn.CrossEntropyLoss`，它内部已经包含了 softmax 和 log 操作，所以我们直接输入 logits 和真实标签的索引即可。

注：cross-entropy loss 期望的输入是 [batch_size * (seq_len-1), vocab_size] 的 output_logits 和 [batch_size * (seq_len-1)] 的 target_ids，所以需要 reshape 一下。

```
# 假设 output_logits 的形状是 [batch_size, seq_len, vocab_size]
# target_ids 的形状是 [batch_size, seq_len]
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(output_logits.reshape(-1, vocab_size), target_ids.reshape(-1))
# -1 表示自动推断维度大小，reshape 括号中填的是新的张量形状
```

2.3 实验

2.3.1 封装

[213]:
```
import matplotlib.pyplot as plt
import seaborn as sns
class AttentionSinkExperiment:
    def __init__(self, num_blocks, corpus=None, num_heads=8, d_model=512,␣
 ↪max_seq_len=4096, learning_rate=3e-4, load_from=None):
        if torch.backends.mps.is_available():
            self.device = torch.device("mps")
        elif torch.cuda.is_available():
            self.device = torch.device("cuda")
        else:
            self.device = torch.device("cpu")
        print(f"Experiment initialized. Using device: {self.device}")
        if load_from is not None:
            # 加载检查点时，corpus 参数不是必需的
            if corpus is not None:
                print("Warning: corpus parameter ignored when loading from␣
 ↪checkpoint")
            # 从检查点加载
            checkpoint = torch.load(load_from, map_location='cpu')
            # 恢复 tokenizer
```

```
            self.tokenizer = SimpleTokenizer('')  # 临时 tokenizer，下面覆盖其内部字典
            self.tokenizer.chars = checkpoint['tokenizer_chars']
            self.tokenizer.char_to_id = checkpoint['tokenizer_char_to_id']
            self.tokenizer.id_to_char = checkpoint['tokenizer_id_to_char']
            self.tokenizer.vocab_size = checkpoint['vocab_size']
            self.vocab_size = checkpoint['vocab_size']
            # 重建模型
            self.model = ToyModel(
                num_blocks=checkpoint['num_blocks'],
                num_heads=checkpoint['num_heads'],
                d_model=checkpoint['d_model'],
                max_seq_len=checkpoint['max_seq_len'],
                vocab_size=checkpoint['vocab_size']
            )
            self.model.load_state_dict(checkpoint['model_state_dict'])
            self.model.to(self.device)
            # 重建优化器
            self.optimizer = torch.optim.Adam(self.model.parameters(), lr=learning_rate)
            self.optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
            # 如果提供了新的学习率，更新优化器中的学习率
            if learning_rate != 3e-4:
                for param_group in self.optimizer.param_groups:
                    param_group['lr'] = learning_rate
            print(f"Model loaded from {load_from}")
        else:
            if corpus is None:
                raise ValueError("corpus parameter is required when load_from is None")
            self.tokenizer = SimpleTokenizer(corpus)
            self.vocab_size = self.tokenizer.vocab_size
            self.model = ToyModel(num_blocks, num_heads, d_model, max_seq_len, self.vocab_size)
            self.model.to(self.device)
```

```
            self.optimizer = torch.optim.Adam(self.model.parameters(),␣
↪lr=learning_rate)
        self.criterion = nn.CrossEntropyLoss()  # 交叉熵损失函数

    def train(self, texts_for_train, epochs=100, log_interval=10,␣
↪save_path=None):
        self.model.train()
        with torch.no_grad():
            original_ids = self.tokenizer.encode(texts_for_train).to(self.
↪device) # [batch_size, seq_len]
            # 根据全部之前的输入预测下一个输出
            input_ids = original_ids[:, :-1]  # 输入序列，去掉最后一个 token
            target_ids = original_ids[:, 1:]  # 目标序列，去掉第一个 token

        for epoch in range(epochs):
            self.optimizer.zero_grad()
            output_logits = self.model(input_ids) # [batch_size, seq_len-1,␣
↪vocab_size]
            # cross-entropy loss 期望的输入是 [batch_size * (seq_len-1),␣
↪vocab_size] 的 output_logits 和 [batch_size * (seq_len-1)] 的 target_ids，所以
需要 reshape
            loss = self.criterion(output_logits.reshape(-1, self.vocab_size),␣
↪target_ids.reshape(-1))
            loss.backward()
            self.optimizer.step()
            if epoch % log_interval == 0:
                print(f"Epoch {epoch}, Loss: {loss.item():.4f}")

        if save_path is not None:
            checkpoint = {
                'model_state_dict': self.model.state_dict(),
                'optimizer_state_dict': self.optimizer.state_dict(),
                'tokenizer_chars': self.tokenizer.chars,
                'tokenizer_char_to_id': self.tokenizer.char_to_id,
                'tokenizer_id_to_char': self.tokenizer.id_to_char,
                'num_blocks': self.model.num_blocks,
```

```
                'num_heads': self.model.num_heads,
                'd_model': self.model.d_model,
                'max_seq_len': self.model.max_seq_len,
                'vocab_size': self.model.vocab_size
            }
            torch.save(checkpoint, save_path)
            print(f"Model saved to {save_path}")

    def visualize_attention(self, text_for_test, layer_idx=-1, head_idx='mean'):
        self.model.eval()
        with torch.no_grad():
            input_ids = self.tokenizer.encode(text_for_test).to(self.device)  #␣
↪[1, seq_len]
            _ = self.model(input_ids)
            attention = self.model.captured_attention[layer_idx]  # [1,␣
↪num_heads, seq_len, seq_len]
            if isinstance(head_idx, int):
                attention_2d = attention[0, head_idx, :, :]  # [seq_len,␣
↪seq_len]
            elif head_idx == 'mean':
                attention_2d = attention.mean(axis=1)[0] # [seq_len, seq_len]

            # 可视化 attention_2d, 例如使用 seaborn 的 heatmap
            tokens = [self.tokenizer.id_to_char[id.item()] for id in␣
↪input_ids[0]]
            plt.figure(figsize=(8, 6))
            sns.heatmap(
                attention_2d,
                cmap="viridis",        # 使用天文/物理界常用的翠绿色调，对数值渐变
非常敏感
                square=True,           # 强制每个单元格为正方形
                xticklabels=tokens,    # X 轴标签（Key）
                yticklabels=tokens,    # Y 轴标签（Query）
                cbar_kws={"shrink": .8}# 缩小一点颜色条，更美观
            )
```

```
            head_title = f"Head {head_idx}" if isinstance(head_idx, int) else␣
↪"Mean of All Heads"
            plt.rcParams['font.sans-serif'] = ['Songti SC']  # 设置中文字体为宋体
            plt.title(f"Attention Map (Layer {layer_idx}, {head_title})")
            plt.xlabel("Key")
            plt.ylabel("Query")
            plt.xticks(rotation=0, ha='right')
            plt.yticks(rotation=0)
            plt.tight_layout()
            plt.show()

    def generate(self, text_for_prompt, max_new_tokens=10):
        self.model.eval()
        with torch.no_grad():
            input_ids = self.tokenizer.encode(text_for_prompt).to(self.device) ␣
↪# [1, seq_len]
            for _ in range(max_new_tokens):
                output_logits = self.model(input_ids)  # [1, seq_len,␣
↪vocab_size]
                next_token_logits = output_logits[:, -1, :]  # [1, vocab_size]
                next_token_logits[:,0] = float('-inf')  # 禁止生成 padding token
                next_token_id = torch.argmax(next_token_logits, dim=-1).
↪unsqueeze(0)  # [1, 1]
                input_ids = torch.cat((input_ids, next_token_id), dim=1)  # 将新
预测的 token id 添加到输入序列末尾
            generated_text = self.tokenizer.decode(input_ids.cpu())  # 解码成文
本
            return generated_text[0]
```

2.3.2 准备工作

[214]:
```
import warnings
import os
warnings.filterwarnings("ignore", category=UserWarning, module="seaborn.utils")␣
↪# 忽略 seaborn 的中文字体警告，避免干扰输出
if __name__ == "__main__":
```

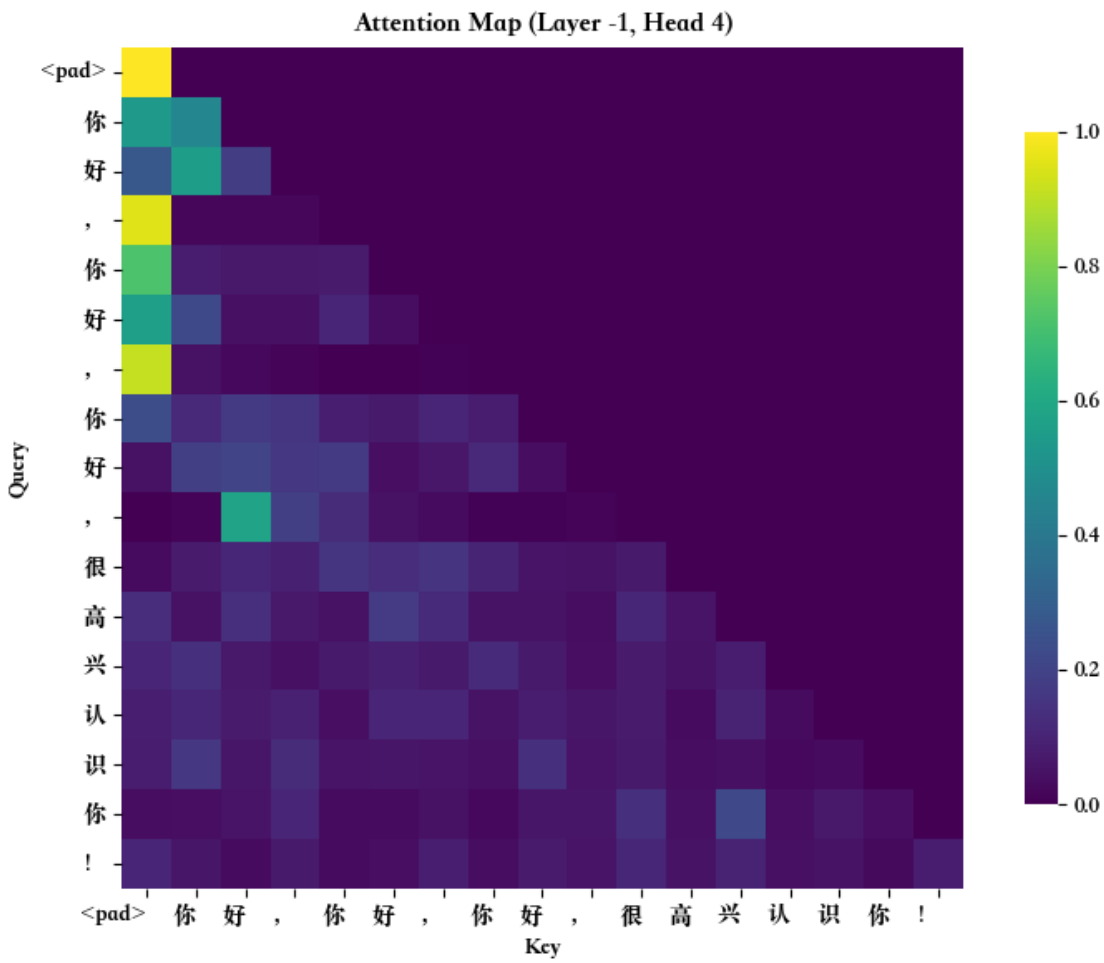
```
# 训练文本：包含重复模式的中文句子，用于凸显 attention sink 现象
texts_for_train = [
    "我我我喜欢学习人工智能。",
    "这个这个这个是一个测试句子。",
    "开始开始开始今天天气很好。",
    "结束结束结束我们回家吧。",
    "啊啊啊这是一个惊讶的表达。",
    "重复重复重复是注意力汇的关键。",
    "你好，你好，你好，很高兴认识你！",
    "今天今天今天是个特殊的日子。",
    "学习学习学习让我进步。",
    "明天明天明天会更好。",
    "人工智能正在改变世界。",
    "注意力机制是 Transformer 模型的核心。",
    "深度学习已经颠覆了许多领域。",
    "自然语言处理让机器理解人类语言。",
    "计算机视觉使机器能够看见。",
    "机器学习算法从数据中学习模式。",
    "神经网络模仿人脑的工作方式。",
    "大数据时代带来了新的挑战和机遇。",
    "云计算提供了强大的计算能力。",
    "物联网连接了物理世界和数字世界。",
    "区块链技术保证了数据的安全性。",
    "量子计算将开启新的计算范式。",
    "自动驾驶汽车正在成为现实。",
    "虚拟现实创造了沉浸式的体验。",
    "增强现实将数字信息叠加到现实世界。",
    "机器人技术正在改变制造业。",
    "基因编辑技术有望治愈遗传疾病。",
    "可再生能源是未来的发展方向。",
    "气候变化是全球性的挑战。",
    "可持续发展是人类共同的目标。",
    "教育是推动社会进步的力量。",
    "文化多样性丰富了人类文明。",
    "艺术表达是人类情感的表现。",
    "体育运动促进身心健康。",
```

```
        "健康饮食有助于维持良好状态。",
        "心理健康与身体健康同样重要。",
        "友谊是人生宝贵的财富。",
        "家庭是温暖的港湾。",
        "爱情是人类永恒的主题。",
        "梦想驱动人们不断前进。",
        "努力工作是成功的基础。",
        "坚持不懈才能克服困难。",
        "创新思维推动社会进步。",
        "团队合作能够实现更大目标。",
        "沟通技巧在人际交往中至关重要。",
        "时间管理提高工作效率。",
        "批判性思维帮助做出明智决策。",
        "终身学习适应快速变化的世界。",
        "感恩之心让生活更加美好。",
        "乐观态度面对生活中的挑战。"
    ]
    # 语料库包含常用汉字、英文字母、数字和标点符号
    corpus = "的一是了我不人在他有这个上们来到时大地为子中你说生国年着就那和要她出也得里后自以会家可下而过天去能对小多然于心学么之都好看起发当没成只如事把还用第样道想作种开美总从无情己面最女但现前些所同日手又行意动方期它头经长儿回位分爱老因很给名法间斯知世什两次使身者被高已亲其进此话常与活正感 1234567890!
 ↪@#$%^&*()_+-=~`qwertyuiopasdfghjklzxcvbnmQWERTYUIOPASDFGHJKLZXCVBNM[]{}|;:
 ↪'\",.<>/? \n,。!?;: “” ‘’ 、 ……——·《》【】 "
    corpus += ''.join(set(''.join(texts_for_train)))  # 确保训练文本中的所有字符都包含在语料库中
    # 如果之前已经训练过模型并保存了检查点，就从检查点继续训练，否则从头开始训练
    if os.path.exists("attention_sink_checkpoint.pth"):
        load_from = "attention_sink_checkpoint.pth"
    else:
        load_from = None
```

2.3.3 实例化并开始实验

[216]:
```
experiment = AttentionSinkExperiment(num_blocks=6, corpus=corpus,␣
↪load_from=load_from,learning_rate=1e-4)
experiment.train(texts_for_train, save_path="attention_sink_checkpoint.pth",␣
↪epochs=20, log_interval=5)
test_text = "你好，你好，你好，很高兴认识你！"
experiment.visualize_attention(test_text, layer_idx=-1, head_idx=4)
print("Generated text:", experiment.generate(test_text, max_new_tokens=100))
```

```
Experiment initialized. Using device: mps
Warning: corpus parameter ignored when loading from checkpoint
Model loaded from attention_sink_checkpoint.pth
Epoch 0, Loss: 0.1701
Epoch 5, Loss: 0.1701
Epoch 10, Loss: 0.1701
Epoch 15, Loss: 0.1701
Model saved to attention_sink_checkpoint.pth
```



Generated text: 你好，你好，你好，很高兴认识你！很高兴认识你！很高兴认识你！础。式。式。式。式。。。

```
        super().__init__(num_heads, d_model, max_seq_len=max_seq_len)
        self.sink_size = sink_size
        self.window_size = window_size
        self.max_cache_len = sink_size + window_size
        self.register_buffer('k_cache', None, persistent=False)
        self.register_buffer('v_cache', None, persistent=False)
        self.seq_len = 0 # 当前已经处理的总序列长度，初始为 0

    def forward(self, x_new):
        batch_size = x_new.shape[0]
        seq_len_new = x_new.shape[1]
        # 无论哪种情况，都需要先计算出纯净的 Q, K, V (借用父类的线性层)
        Q_new = self.W_Q(x_new).view(batch_size, seq_len_new, self.num_heads,␣
↪self.d_k).transpose(1, 2)
        K_new = self.W_K(x_new).view(batch_size, seq_len_new, self.num_heads,␣
↪self.d_k).transpose(1, 2)
        V_new = self.W_V(x_new).view(batch_size, seq_len_new, self.num_heads,␣
↪self.d_k).transpose(1, 2)
        # 初始化预分配固定内存 (Ring Buffer)
        if self.k_cache is None or self.k_cache.shape[0] != batch_size:
            self.k_cache = torch.zeros((batch_size, self.num_heads, self.
↪max_cache_len, self.d_k), device=x_new.device, dtype=K_new.dtype)
            self.v_cache = torch.zeros_like(self.k_cache)
            self.seq_len = 0
        # ============================================================
        # 分成两大阶段: 预填充阶段 (prefill) 和流式解码阶段 (decoding)
        if seq_len_new>1:
            # 预填充阶段 (prefill), 输入一段长提示词
            if seq_len_new <= self.max_cache_len:
                ### 还没超长，全部存下
                self.k_cache[:, :, :seq_len_new, :] = K_new
                self.v_cache[:, :, :seq_len_new, :] = V_new
            else:
                ### 超长了，存满 sink_size 的部分，剩余的部分只保留最后␣
↪window_size 的部分
```

```
                self.k_cache[:, :, :self.sink_size, :] = K_new[:, :, :self.
↪sink_size, :]
                self.v_cache[:, :, :self.sink_size, :] = V_new[:, :, :self.
↪sink_size, :]
                self.k_cache[:, :, self.sink_size:, :] = K_new[:, :, -self.
↪window_size:, :]
                self.v_cache[:, :, self.sink_size:, :] = V_new[:, :, -self.
↪window_size:, :]
            ## 更新当前序列长度
            self.seq_len = seq_len_new
            ## 按照父类的逻辑继续计算注意力
            Q_rotated = self.rope(Q_new)
            K_rotated = self.rope(K_new)
            ### 缩放点积注意力与因果掩码 Causal Masking
            #### Query-Key 点积计算注意力得分
            scores = torch.matmul(Q_rotated, K_rotated.transpose(-2, -1)) /␣
↪(self.d_k ** 0.5)  # [batch_size, num_heads, seq_len, seq_len]
            #### 因果掩码和 softmax 计算注意力权重
            Attention = F.softmax(scores + self.mask[..., :seq_len_new, :
↪seq_len_new], dim=-1)  # [batch_size, num_heads, seq_len, seq_len]
            self.captured_attention = Attention  # ** 新增 **: 捕获注意力矩阵，用
hook 记录下来，方便后续分析
            #### 乘以 Value 完成加权求和
            Out = torch.matmul(Attention, V_new)  # [batch_size, num_heads,␣
↪seq_len, d_k]
            ### 多头拼接并乘以输出矩阵 (必须先内存连续化 (contiguous) 再 view，否
则会报错)
            Out = Out.transpose(1, 2).contiguous().view(batch_size,␣
↪seq_len_new, -1)  # [batch_size, seq_len, d_model]
            H = self.W_O(Out)  # [batch_size, seq_len, d_model]
            return H
        # ============================================================
        else:
            # 流式解码阶段 (decoding)，每次输入一个新 token
            ## 计算插入索引并原位更新
```

```
            if self.seq_len < self.max_cache_len:
                insert_idx = self.seq_len
            else:
                insert_idx = self.sink_size + (self.seq_len-self.sink_size) %␣
↪self.window_size
            self.k_cache[:, :, insert_idx:insert_idx+1, :] = K_new
            self.v_cache[:, :, insert_idx:insert_idx+1, :] = V_new
            ## 提取出当前有效的 Cache 进行 Attention 计算
            if self.seq_len < self.max_cache_len:
                K_active = self.k_cache[:, :, :self.seq_len+1, :]
                V_active = self.v_cache[:, :, :self.seq_len+1, :]
            else:
                K_active = torch.cat((self.k_cache[:, :, :self.sink_size, :],␣
↪self.k_cache[:, :, insert_idx+1:, :], self.k_cache[:, :, self.sink_size:
↪insert_idx+1, :]), dim=2)
                V_active = torch.cat((self.v_cache[:, :, :self.sink_size, :],␣
↪self.v_cache[:, :, insert_idx+1:, :], self.v_cache[:, :, self.sink_size:
↪insert_idx+1, :]), dim=2)
            self.seq_len += 1
            ## RoPE 位置编码
            K_active = self.rope(K_active)
            pad_len = K_active.shape[2]
            dummy_Q = torch.zeros((batch_size, self.num_heads, pad_len, self.
↪d_k), device=x_new.device, dtype=x_new.dtype)
            dummy_Q[:, :, -1:, :] = Q_new  # 只在最后一个位置放入新计算的 Q，其他
位置全是 0，这样就能正确地应用 RoPE 位置编码
            dummy_Q = self.rope(dummy_Q)
            Q_new = dummy_Q[:, :, -1:, :]  # 提取出最后一个位置的 Q，形状仍然是␣
↪[batch_size, num_heads, 1, d_k]
            ## 缩放点积注意力
            ### 单步生成时，不需要 Causal Mask! 因为所有的 K 都是历史或现在，根本不
存在未来的信息泄露问题了。
            scores = torch.matmul(Q_new, K_active.transpose(-2, -1)) / self.d_k␣
↪** 0.5
            Attention = F.softmax(scores, dim=-1)
            self.captured_attention = Attention
```

```
        Out = torch.matmul(Attention, V_active)
        Out = Out.transpose(1, 2).contiguous().view(batch_size, 1, -1)
        H = self.W_O(Out)
        return H
```

用上述的 `StreamingMultiHeadAttention` 模块替换原来的 `MultiHeadAttention` 模块即可

4 模型性能指标

4.1 困惑度 (Perplexity)

4.1.1 理论公式

假设一段由 N 个 Token 组成的文本序列为 $X = (x_1, x_2, ..., x_N)$，模型基于前面的 Token 预测下一个 Token 的概率为 $P(x_i|x_{<i})$。那么这段文本的困惑度定义为：

$$PPL(X) = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(x_i|x_{<i})\right)$$

如果模型对每个 Token 的预测概率都很高，那么困惑度就接近 1，表示模型非常自信；如果预测概率较低，困惑度就会很大，表示模型不确定。

当词表大小为 V，且模型对每个 Token 的预测概率均匀分布时，困惑度达到最大值 $PPL_{max} = V$。因此，困惑度的范围是 $[1, V]$，数值越小表示模型性能越好。

如果每个 Token 的预测正确概率都为 99%，那么困惑度就是：

$$PPL = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log 0.99\right) \approx 1.01$$